

1 LabLink: Cloud-based virtual teaching lab accessible 2 through the browser

3 **Andrew Park** ¹, **Elizabeth Berrigan** ¹, **Liezl Maree** ¹, and **Talmo**
4 **Pereira** ¹¶

5 ¹ Salk Institute for Biological Studies, La Jolla, California, United States ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#)).

6 Summary

7 Quantitative biology increasingly relies on GPU-accelerated deep learning tools with graphical
8 interfaces, but teaching them hands-on is hard: installing complex software environments
9 consumes the opening hours of a workshop, and many participants' laptops have no GPU at
10 all. LabLink is an open-source Infrastructure-as-Code (IaC) platform with which a research
11 group can deploy and manage reproducible GPU computing environments in its own Amazon
12 Web Services (AWS) account or on hardware it already owns. A lightweight allocator service
13 manages one machine per participant, provisioned in the cloud or registered from the lab's
14 own hardware, and monitors its health; each workshop participant claims a seat by entering
15 their email on a web page and lands in a full Linux desktop with the instructor's software
16 preinstalled, running in a browser tab ([Figure 1](#)). LabLink removes per-participant installation
17 entirely: in our benchmark, a 30-seat pool was ready about seven minutes after a cold start.
18 The platform has supported approximately 600 users across 15 events, from K–12 students to
19 faculty researchers.

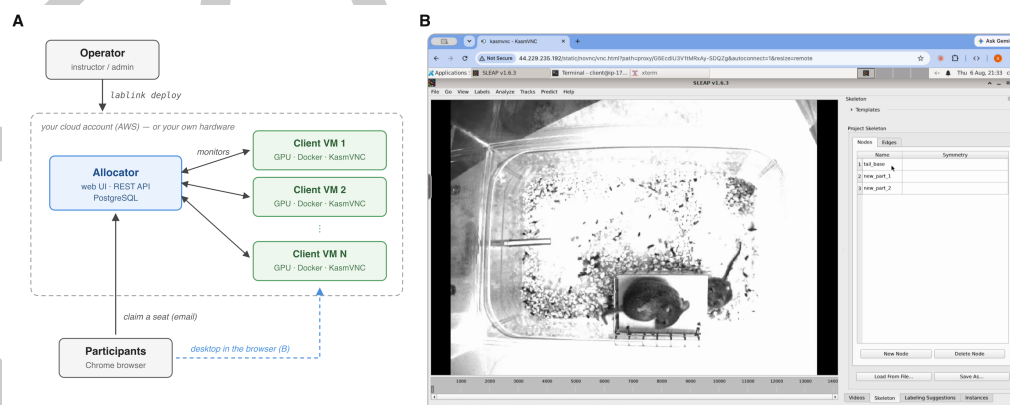


Figure 1: LabLink overview. (A) An operator deploys the allocator into their own cloud account or onto a lab-owned server with a single CLI command; the allocator manages one client VM per participant (provisioned on AWS, or registered bring-your-own machines) and monitors their health. Participants enter an email address and receive a full Linux desktop streamed to their browser. (B) The participant's view: SLEAP v1.6.3 running in a LabLink browser desktop.

20 Statement of need

21 Hands-on workshops are how research software spreads ([Wilson, 2016](#)), but GPU-dependent
22 scientific software is hard to teach. Installation means resolving complex dependency stacks,

GPU drivers, and operating-system-specific configuration (Mangul et al., 2019); a classroom presents a different failure mode per laptop, and the first hours of a workshop are routinely lost to setup. Worse, many participants have no local GPU, so even a successful installation leaves them unable to run training or inference.

The common workarounds each sacrifice something essential. Asking participants to preinstall shifts the burden onto the least experienced users. Shared teaching servers require institutional infrastructure and maintenance effort most labs do not have, and cloud notebooks solve hardware access but cannot host interactive graphical applications. LabLink closes this gap: a research lab can set up a group of identical, isolated, GPU-backed Linux desktops in its own cloud account, hand each participant one link, and tear everything down when the workshop ends. LabLink targets maintainers and instructors of GPU-dependent scientific software who need to give a room of participants working environments without per-laptop setup or institutional computing infrastructure.

State of the field

Domain platforms such as brainlife.io (Hayashi et al., 2024), Neurodesk (Renton et al., 2024), and brainchop (Masoud et al., 2023) deliver cloud-based or in-browser neuroimaging environments. NeuroCAAS (Abe et al., 2022) applies infrastructure-as-code to neuroscience but serves reproducible analyses as batch jobs rather than interactive desktops. These serve their target domains well, but each is organized around its own application catalog rather than around software the instructor packages themselves.

General-purpose teaching infrastructure is notebook-centric. JupyterHub (Project Jupyter contributors, 2026a) can stream a Linux desktop through extensions such as jupyter-remote-desktop-proxy (Project Jupyter contributors, 2026b), but this is a standing service that someone must keep running. A small class can run one on a single server with The Littlest JupyterHub (Project Jupyter contributors, 2026c). Larger classes, and BinderHub (Project Jupyter et al., 2018), need a Kubernetes cluster. Providers such as 2i2c (2i2c, 2026) will host the hub instead. None of them automates the event itself: adding GPU machines, admitting participants, and tearing everything down are left to the operator of a persistent hub. Browser development environments such as GitHub Codespaces, Gitpod, and Coder offer editors and terminals rather than GPU desktops for graphical applications. The Galaxy project's Training Infrastructure as a Service (Rasche et al., 2023) has supported over 500 training events with more than 24,000 learners but serves Galaxy workflows on shared public servers, not an instructor's own software. Google Colab offers GPUs without operational burden, but imposes session limits, supports only notebooks, and gives the instructor no control over the environment.

Browser-based remote desktops themselves are well established: Apache Guacamole (The Apache Software Foundation, 2026) gateways a browser to existing machines, Open OnDemand (Hudak et al., 2018) delivers graphical desktops from an institutional HPC cluster with center-level administration, and Kasm Workspaces (Kasm Technologies, 2026a) orchestrates containerized desktops as a standing, account-based service that an administrator installs and maintains.

Cloud vendors also serve this niche, as standing, account-based services. Azure Lab Services (Microsoft, 2026) provisions per-student VM pools with scheduled start and stop, but has been closed to new customers since July 2024 and retires in June 2027. Amazon WorkSpaces and WorkSpaces Applications (formerly AppStream 2.0) (Amazon Web Services, 2026b) stream managed, GPU-capable desktops and applications from an AWS account. Both are also bound to their vendor's cloud, whereas a LabLink deployment can run entirely on GPU machines the lab already owns.

LabLink's contribution is therefore not the browser desktop but the integration around it: reproducible provisioning of one GPU machine per participant in infrastructure the operator

owns, desktops running instructor-packaged software, countless email-based seat claiming, real-time health monitoring with automatic recovery, and teardown when the event ends. We wrote new software because the missing piece is the workshop lifecycle itself, not the desktop or notebook underneath it: existing platforms are built as standing services an administrator maintains, whereas a LabLink deployment is created for one event and destroyed afterward.

Software design

LabLink's architecture centers on two components: an **allocator** that manages a group of client machines, and a **client service** that runs on each machine. The allocator is a Flask ([Pallets, 2026](#)) service that owns a PostgreSQL ([The PostgreSQL Global Development Group, 2026](#)) database of client machines (cloud VMs it provisions by running OpenTofu ([OpenTofu contributors, 2026](#)), an open-source Terraform-compatible IaC tool, as asynchronous background operations, or machines that the lab owns) and assigns each participant who claims a seat to an unclaimed machine. It also serves as an admin dashboard with per-machine status and logs, and can destroy client machines at a scheduled time so a forgotten pool does not outlive its event. Each client machine runs one Docker ([Merkel, 2014](#)) container built from the instructor's image, which hosts both the KasmVNC ([Kasm Technologies, 2026b](#)) desktop the participant sees and the client service reporting the machine's state. Giving each participant a whole machine rather than a slice of a shared server trades infrastructure cost for isolation (one participant's crashed session or saturated GPU affects only their own seat). At AWS on-demand prices in us-west-2 ([Amazon Web Services, 2026a](#)), a g4dn.xlarge seat costs about \$0.53 per hour, so the compute for a four-hour, 30-seat workshop comes to about \$63, roughly \$2 per participant. The two components communicate over a simple contract: the client service registers its machine with the allocator, then reports heartbeats, GPU health, and seat status through client-initiated REST API calls. The client machine's KasmVNC session is streamed over WebSockets and proxied through the allocator's nginx ([nginx contributors, 2026](#)), making the allocator the deployment's single participant-facing endpoint.

Deployment

Deployment is deliberately decoupled from these components: the OpenTofu definition of the allocator's infrastructure lives in a separate template repository (<https://github.com/talmolab/lablink-template>). Operators deploy either through the CLI (`lablink-cli`) — an interactive configuration wizard plus commands to provision and monitor machines — or, for version-controlled, reviewable infrastructure changes, by forking the template repository and using its GitHub Actions workflows. Both paths deploy the same allocator image and infrastructure definition. All three packages ship automated test suites that run in continuous integration on every change.

The manual provider extends the same client-allocator contract to bring-your-own (BYO) machines, such as lab workstations or containers on an institutional GPU scheduler. Because all control traffic is outbound HTTP and the allocator itself can be published through an outbound tunnel, neither side needs inbound firewall rules, and several desktop transport options (direct LAN connection, a WireGuard-based mesh overlay, or a client-initiated WebSocket tunnel) cover firewalled and fully on-premises topologies.

Security

The security model follows from this topology. The allocator is the only participant-facing endpoint, and desktops are reached only through its proxy. TLS is a configuration option (Let's Encrypt, Cloudflare, or AWS Certificate Manager), not a default; a deployment reached by bare IP address runs over plain HTTP. Admin pages and operator APIs require operator credentials; client machines authenticate with per-machine secrets minted at registration and never hold database or admin credentials.

The seat-claim page is deliberately unauthenticated: an email address labels a seat rather than authenticating a user, and no passcode or rate limit gates it. This is acceptable only because a deployment is short-lived with a fixed pool. A leaked link can exhaust seats but cannot provision machines, one email address holds at most one seat, and scheduled destruction bounds the deployment's lifetime. Claiming a seat rotates that desktop's password and binds the browser to a signed session cookie, so participants cannot open one another's seats. Participants have administrative rights inside their desktop container but not on the host, and on AWS a client machine accepts no inbound connections other than operator SSH. The address is format-checked but never verified, so any email-shaped token serves as a seat label; it is held only within the deployment (its database and service logs) and is destroyed with it.

Adaptability

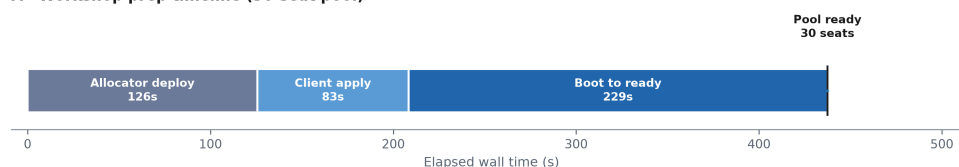
Adapting a deployment to new software means changing two files: a YAML configuration that names the Docker image, and a startup script that installs the instructor's software and stages tutorial data as the client container boots.

Benchmark

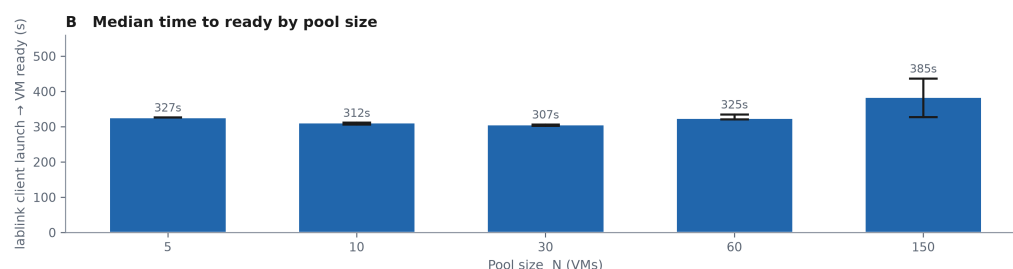
Workshop-scale behavior is measured in Figure 2. From a cold start, a ready 30-seat pool takes about seven minutes. In single runs at pool sizes of 5, 10, 30, and 60 VMs, the per-VM median time from the pool-launch command to a VM ready to be assigned stayed around five minutes. A 150-seat run, the size of the largest event in Table 1, had all 150 VMs ready 11 minutes after the launch command, with a per-VM median of about six and a half minutes; its infrastructure apply took 249 s against 83 s at 30 seats, so the apply phase grows with pool size while per-VM boot time grows only modestly.

Workshop-scale benchmark (5–150 seats)

A Workshop prep timeline (30-seat pool)



B Median time to ready by pool size



C VM outcomes by pool size

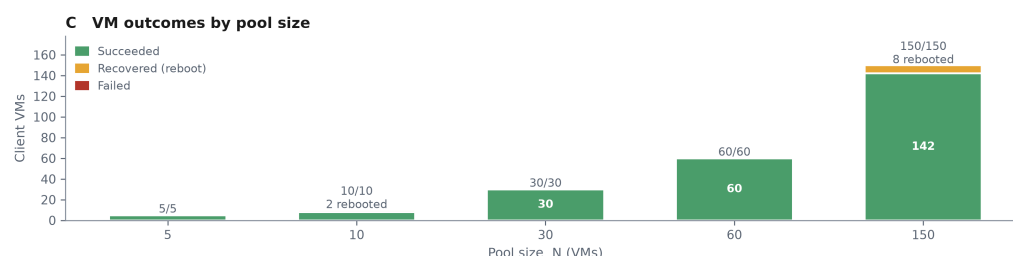


Figure 2: Workshop-scale benchmark: a LabLink deployment scaled from 5 to 150 seats on g4dn.xlarge instances (us-west-2), one run per pool size. The pools of 5 to 60 seats share one pinned client-image digest; the 150-seat pool ran twelve days later on the 0.4.0 release image. (A) Wall-clock to prepare one 30-seat workshop: allocator deploy (126 s), client-VM apply (83 s), and boot until all 30 VMs were ready (229 s) — a ready pool in about 7 min from nothing. (B) Per-VM readiness time by pool size N, measured from the operator’s single pool-launch command until the allocator marks each VM ready to be assigned; bars are medians across the pool’s VMs, whiskers the interquartile range (IQR) across those VMs within the run, not across runs. B excludes the one-time allocator deploy. (C) VM outcomes per pool: the number of VMs that reached ready directly, after one automatic reboot, or failed. All 255 VMs across the five pools reached ready; ten required one automatic reboot (two at N = 10, eight at N = 150), exercising the recovery path. The figure script, run metadata, and raw data are in `paper/fig2-harness/` in the LabLink repository.

Research impact statement

LabLink was built to run SLEAP (Pereira et al., 2022) workshops and has since delivered on-demand GPU desktops at 15 events with approximately 600 participants between 2024 and 2026 (Table 1), spanning audiences from K–12 students to graduate students and faculty.

LabLink’s contribution to research runs through dissemination. Installation is a documented barrier to adopting research software (Mangul et al., 2019); for GPU-dependent tools, a hands-on workshop is often a researcher’s first opportunity to try the software at all. By removing setup for an entire classroom at once, LabLink converts the hours a workshop would otherwise spend on installation into supervised practice of the complete GPU workflow, on hardware every participant can run it on, with the instructor present when something goes wrong.

This generalizes beyond SLEAP. LabLink environments for Cellpose (Stringer et al., 2021), Kilosort4 (Pachitariu et al., 2024), idtracker.ai (Romero-Ferrero et al., 2019), and DeepLabCut

(Mathis et al., 2018) each required changing only the configuration and startup script, and any group that maintains a GPU-dependent research tool can hand its users a working GPU desktop the same way.

Table 1: Workshops and courses delivered on LabLink, 2024–2026. Participant counts marked ~ are organizer estimates.

Event	Date(s)	Participants	Location
Cosyne Main Tutorial – SLEAP	Feb 29, 2024	~150	Lisbon, Portugal
University of Illinois Urbana–Champaign workshop	Apr 2, 2024	~40	Urbana, IL, USA
Keiller Leadership Academy STEM Day	May 30, 2024	~125 (K–12 students)	San Diego, CA, USA
CAJAL Advanced Neuroscience Training Programme	Jun 2024	~20	Champalimaud Centre for the Unknown, Lisbon, Portugal
Salk EDGE Summer Program	Jul 24, 2024	3	La Jolla, CA, USA
NYU Langone Neuroscience T32 Workshop	Aug 19, 2024	~25	New York, NY, USA
Ellen Potter Research Connections	Apr 9–10, 2025	~20	Salk Institute, La Jolla, CA, USA
Bridges to the Future Symposium	May 28, 2025	~50	Salk Institute, La Jolla, CA, USA
CAJAL Advanced Neuroscience Training Programme	Jun 8–27, 2025	~30	Champalimaud Centre for the Unknown, Lisbon, Portugal
Zagreb NeuroData Summer School	Jul 2025	~30	Online / Zagreb, Croatia
Salk EDGE Summer Program	Jul 14–15, 2025	2	Salk Institute, La Jolla, CA, USA
Computer Vision for Biologists	Sep 1–12, 2025	~30	University of California Davis, CA, USA
Introduction to Multi-Animal Pose Tracking (US-RSE 2025)	Oct 6, 2025	~30	Philadelphia, PA, USA
Short course on machine learning applications	Oct 13–17, 2025	~30	Jackson Laboratory, Bar Harbor, ME, USA
CAJAL Advanced Neuroscience Training Programme	Jun 10, 2026	18	Champalimaud Centre for the Unknown, Lisbon, Portugal

AI usage disclosure

Claude Opus 5 (Anthropic) was used in developing both the software and this manuscript. The architecture and initial implementation of LabLink were designed and implemented by the human authors. Claude Code was later used to draft code and documentation. The paper was drafted by the human authors, and Claude Fable 5 was used to review the text and to help assemble metrics and figures. All generated code, documentation, and figures were reviewed by the human authors and validated with the automated test suites and in live deployments.

Acknowledgements

This work was supported by the NIH/BRAIN Initiative RF1 award under grant number 1RF1MH132653-01. We want to acknowledge the open-source projects that made this work possible, including OpenTofu, Docker, KasmVNC, and PostgreSQL.

References

- 2i2c. (2026). *2i2c: Managed JupyterHub infrastructure for research and education*. <https://2i2c.org/>
- Abe, T., Kinsella, I., Saxena, S., Buchanan, E. K., Couto, J., Briggs, J., Kitt, S. L., Glassman, R., Zhou, J., Paninski, L., & Cunningham, J. P. (2022). Neuroscience cloud analysis as a service: An open-source platform for scalable, reproducible data analysis. *Neuron*, 110(17), 2771–2789.e7. <https://doi.org/10.1016/j.neuron.2022.06.018>
- Amazon Web Services. (2026a). *Amazon EC2 on-demand pricing*. <https://aws.amazon.com/ec2/pricing/on-demand/>
- Amazon Web Services. (2026b). *Amazon WorkSpaces and amazon WorkSpaces applications (formerly AppStream 2.0)*. <https://aws.amazon.com/workspaces/>
- Hayashi, S., Caron, B. A., Heinsfeld, A. S., Vinci-Booher, S., McPherson, B., Bullock, D. N., Bertò, G., Niso, G., Hanekamp, S., Levitas, D., Ray, K., MacKenzie, A., Avesani, P., Kitchell, L., Leong, J. K., Nascimento-Silva, F., Koudoro, S., Willis, H., Jolly, J. K., ... Pestilli, F. (2024). Brainlife.io: A decentralized and open-source cloud platform to support neuroscience research. *Nature Methods*, 21, 809–813. <https://doi.org/10.1038/s41592-024-02237-2>
- Hudak, D., Johnson, D., Chalker, A., Nicklas, J., Franz, E., Dockendorf, T., & McMichael, B. L. (2018). Open OnDemand: A web-based client portal for HPC centers. *Journal of Open Source Software*, 3(25), 622. <https://doi.org/10.21105/joss.00622>
- Kasm Technologies. (2026a). *Kasm workspaces*. <https://kasmweb.com>
- Kasm Technologies. (2026b). *KasmVNC: Modern VNC server and client, web based and secure*. <https://github.com/kasmtech/KasmVNC>
- Mangul, S., Mosqueiro, T., Abdill, R. J., Duong, D., Mitchell, K., Sarwal, V., Hill, B., Brito, J., Littman, R. J., Statz, B., Lam, A. K.-M., Dayama, G., Grieneisen, L., Martin, L. S., Flint, J., Eskin, E., & Blekhman, R. (2019). Challenges and recommendations to improve the installability and archival stability of omics computational tools. *PLOS Biology*, 17(6), e3000333. <https://doi.org/10.1371/journal.pbio.3000333>
- Masoud, M., Hu, F., & Plis, S. (2023). Brainchop: In-browser MRI volumetric segmentation and rendering. *Journal of Open Source Software*, 8(83), 5098. <https://doi.org/10.21105/joss.05098>
- Mathis, A., Mamidanna, P., Cury, K. M., Abe, T., Murthy, V. N., Mathis, M. W., & Bethge, M. (2018). DeepLabCut: Markerless pose estimation of user-defined body parts with deep learning. *Nature Neuroscience*, 21(9), 1281–1289. <https://doi.org/10.1038/s41593-018-0209-y>
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239), 2.
- Microsoft. (2026). *Azure lab services retirement guide*. <https://learn.microsoft.com/en-us/azure/lab-services/retirement-guide>
- nginx contributors. (2026). *Nginx*. <https://nginx.org>
- OpenTofu contributors. (2026). *OpenTofu: The open source infrastructure as code tool*.

- 209 <https://opentofu.org>
- 210 Pachitariu, M., Sridhar, S., Pennington, J., & Stringer, C. (2024). Spike sorting with Kilosort4.
211 *Nature Methods*, 21(5), 914–921. <https://doi.org/10.1038/s41592-024-02232-7>
- 212 Pallets. (2026). *Flask*. <https://flask.palletsprojects.com>
- 213 Pereira, T. D., Tabris, N., Matsliah, A., Turner, D. M., Li, J., Ravindranath, S., Papadoyannis,
214 E. S., Normand, E., Deutsch, D. S., Wang, Z. Y., McKenzie-Smith, G. C., Mitelut, C. C.,
215 Castro, M. D., D'Uva, J., Kislin, M., Sanes, D. H., Kocher, S. D., Wang, S. S.-H., Falkner,
216 A. L., ... Murthy, M. (2022). SLEAP: A deep learning system for multi-animal pose tracking.
217 *Nature Methods*, 19(4), 486–495. <https://doi.org/10.1038/s41592-022-01426-1>
- 218 Project Jupyter contributors. (2026a). *JupyterHub: Multi-user server for jupyter notebooks*.
219 <https://github.com/jupyterhub/jupyterhub>
- 220 Project Jupyter contributors. (2026b). *Jupyter-remote-desktop-proxy: Run a linux desktop on*
221 *a JupyterHub*. <https://github.com/jupyterhub/jupyter-remote-desktop-proxy>
- 222 Project Jupyter contributors. (2026c). *The littlest JupyterHub*. <https://github.com/jupyterhub/the-littlest-jupyterhub>
- 223
- 224 Project Jupyter, Bussonnier, M., Forde, J., Freeman, J., Granger, B., Head, T., Holdgraf,
225 C., Kelley, K., Nalvarte, G., Osheroff, A., Pacer, M., Panda, Y., Pérez, F., Ragan-Kelley,
226 B., & Willing, C. (2018). Binder 2.0 – reproducible, interactive, sharable environments
227 for science at scale. *Proceedings of the 17th Python in Science Conference*, 113–120.
228 <https://doi.org/10.25080/Majora-4af1f417-011>
- 229 Rasche, H., Hyde, C., Davis, J., Gladman, S., Coraor, N., Bretaudeau, A., Cuccuru, G.,
230 Bacon, W., Serrano-Solano, B., Hillman-Jackson, J., Hiltmann, S., Zhou, M., Grüning,
231 B., & Stubbs, A. (2023). Training infrastructure as a service. *GigaScience*, 12, giad048.
232 <https://doi.org/10.1093/gigascience/giad048>
- 233 Renton, A. I., Dao, T. T., Johnstone, T., Civier, O., Sullivan, R. P., White, D. J., Lyons, P.,
234 Slade, B. M., Abbott, D. F., Amos, T. J., Bollmann, S., Botting, A., Campbell, M. E.
235 J., Chang, J., Close, T. G., Dörig, M., Eckstein, K., Egan, G. F., Evas, S., ... Bollmann,
236 S. (2024). Neurodesk: An accessible, flexible and portable data analysis environment
237 for reproducible neuroimaging. *Nature Methods*, 21, 804–808. <https://doi.org/10.1038/s41592-023-02145-x>
- 238
- 239 Romero-Ferrero, F., Bergomi, M. G., Hinz, R. C., Heras, F. J. H., & Polavieja, G. G. de.
240 (2019). Idtracker.ai: Tracking all individuals in small or large collectives of unmarked
241 animals. *Nature Methods*, 16(2), 179–182. <https://doi.org/10.1038/s41592-018-0295-5>
- 242 Stringer, C., Wang, T., Michaelos, M., & Pachitariu, M. (2021). Cellpose: A generalist
243 algorithm for cellular segmentation. *Nature Methods*, 18(1), 100–106. <https://doi.org/10.1038/s41592-020-01018-x>
- 244
- 245 The Apache Software Foundation. (2026). *Apache guacamole*. <https://guacamole.apache.org>
- 246 The PostgreSQL Global Development Group. (2026). *PostgreSQL*. <https://www.postgresql.org>
- 247 Wilson, G. (2016). Software carpentry: Lessons learned. *F1000Research*, 3, 62. <https://doi.org/10.12688/f1000research.3-62.v2>
- 248